

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear:2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week3 - Wednesday	Time(s)	
Duration	2 Hours	Applicableto Batches	
AssignmentNumber:6.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	ExpectedTime to complete	
1	Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals Lab Objectives: - To explore AI-powered auto-completion features for core Python constructs. - To analyze how AI suggests logic for class definitions, loops, and conditionals. - To evaluate the completeness and correctness of code generated by AI assistants. Lab Outcomes (LOs):	Week3 - Wednesday	

After completing this lab, students will be able to:

- Use AI tools to generate and complete class definitions and methods.
- Understand and assess AI-suggested loops for iterative tasks.
- Generate conditional statements through prompt-driven suggestions.
- Critically evaluate AI-assisted code for correctness and clarity.

Task Description#1 (Classes)

- Use AI to complete a Student class with attributes and a method.
- Check output
- Analyze the code generated by AI tool

Instructions:

- **Initialize class with attributes like name, roll no, marks**
- **Method to display student details**
- **Method to calculate grade based on marks (A: >=90, B: >=75, C: >=60, else Fail)**

Start Writing code and auto complete using any AI tool

Expected Output#1

- Class with constructor and display_details() method

Task Description#2 (Loops)

- Prompt AI to complete a function that prints the first 10 multiples of a number using a loop.
- Analyze the generated code
- Ask AI to generate code using other controlled looping

Write code using **For** Loop, later complete code using **While** Loop

Expected Output#2

- Correct loop-based implementation

Task Description#3 (Conditional Statements)

- Ask AI to write nested if-elif-else conditionals to classify age groups.
- Analyze the generated code
- Ask AI to generate code using other conditional statements

Table: Age Group Classification Logic

Age Range	Age Group
0 – 12 years	Child
13 – 19 years	Teen
20 – 59 years	Adult
60 years & above	Senior

Expected Output#3

- Age classification function with appropriate conditions and with explanation

Task Description#4 (For and While loops)

- Generate a sum_to_n() function to calculate sum of first n numbers
- Analyze the generated code
- Get suggestions from AI with other controlled looping

Expected Output#4

- Python code with explanation

TASK 1:

PROMPT: class Student:

```
def __init__(self, name, roll_no, marks):
    self.name = name
    self.roll_no = roll_no
    self.marks = marks
```

Generate the python code with calling student class and methods and give grades according to (A:>=90,B:>=75,C:>=60,else fail.)

CODE:

```
task1.py X
task1.py > ...
1  class Student:
7      def get_grade(self):
10         elif self.marks >= 75:
11             return 'B'
12         elif self.marks >= 60:
13             return 'C'
14         else:
15             return 'Fail'
16
17     def display(self):
18         print(f"Name: {self.name}")
19         print(f"Roll No: {self.roll_no}")
20         print(f"Marks: {self.marks}")
21         print(f"Grade: {self.get_grade()}")
22
23     # Example usage:
24     student1 = Student("Alice", 101, 92)
25     student2 = Student("Bob", 102, 78)
26
27     student1.display()
28     print()
29     student2.display()
30     print()
```

Output:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Nami\OneDrive\Desktop\ai\lab 6> & C:/Users/Nami/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/Nami/OneDrive/Desktop/ai/lab 6/task1.py"
Name: Alice
Roll No: 101
Marks: 92
Grade: A

Name: Bob
Roll No: 102
Marks: 78
Grade: B

PS C:\Users\Nami\OneDrive\Desktop\ai\lab 6>
```

TASK 2:

PROMPT : def print_multiples(number):

for i in range(1, 11):

print(number * i)

now generate this code in while loop.

CODE :

```
task1.py task2.py X
task2.py > ...
1 def print_multiples(number):
2     i = 1
3     while i <= 10:
4         print(number * i)
5         i += 1
6 print_multiples(10)
```

Output:

```
Problems Output Debug Console Terminal Ports
Python + - [ ] [ ] ... ^ x
PS C:\Users\Nami\OneDrive\Desktop\ai\lab 6> & C:/Users/Nami/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/Nami/OneDrive/Desktop/ai/lab 6/task2.py"
10
20
30
40
50
60
70
80
90
100
PS C:\Users\Nami\OneDrive\Desktop\ai\lab 6>
```

TASK 3:

PROMPT:

```
def classify_age(age):
```

```
    if age >= 0:
```

```
        if age <= 12:
```

```
            print("Child")
```

```
        elif age <= 17:
```

```
            print("Teenager")
```

```
        elif age <= 64:
```

```
            print("Adult")
```

```
        else:
```

```
            print("Senior")
```

```
    else:
```

```
        print("Invalid age")
```

now generate the same code using another conditional statements.

CODE :

```
task1.py task2.py task3.py X
task3.py > ...
1 def classify_age(age):
2     match True:
3         case _ if age < 0:
4             print("Invalid age")
5         case _ if age <= 12:
6             print("child")
7         case _ if age <= 17:
8             print("Teenager")
9         case _ if age <= 64:
10            print("Adult")
11        case _:
12            print("Senior")
13    classify_age(25)
```

Output:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Namitha\OneDrive\Desktop\ai\lab 6> & C:/Users/Namitha/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/Namitha/OneDrive/Desktop/ai/lab 6/task3.py"
Adult
PS C:\Users\Namitha\OneDrive\Desktop\ai\lab 6>
```

TASK 4:

PROMPT: def sum_to_n(n):

total = 0

for i in range(1, n + 1):

total += i

return total

Now generate the same code with while loop.

CODE:

```
task1.py task2.py task3.py task4.py X
task4.py > ...
1 def sum_to_n(n): | TAB to jump here
2     total = 0
3     i = 1
4     while i <= n:
5         total += i
6         i += 1
7     return total
8 print(sum_to_n(10))
```

Output:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Nami\OneDrive\Desktop\ai\lab 6> & C:/Users/Nami/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/Nami/OneDrive/Desktop/ai/lab 6/task4.py"
55
PS C:\Users\Nami\OneDrive\Desktop\ai\lab 6>
```

TASK 5:

PROMPT: class BankAccount:

```
def __init__(self, owner, balance=0):
    self.owner = owner
    self.balance = balance
```

```
def deposit(self, amount):
    if amount > 0:
        self.balance += amount
        print(f"Deposited ${amount}")
    else:
        print("Deposit amount must be positive.")
```

```
def withdraw(self, amount):
    if amount <= self.balance:
        self.balance -= amount
        print(f"Withdrew ${amount}")
    else:
        print("Insufficient funds.")
```

```
def get_balance(self):
    return self.balance
```

Now explain this code using comments.

CODE:

```
task1.py task2.py task3.py task4.py task5.py X
task5.py > ...
1  # This class represents a simple bank account.
2  class BankAccount:
3      def __init__(self, owner, balance=0):
4          # Initialize the account with the owner's name and an optional starting balance (default is 0)
5          self.owner = owner
6          self.balance = balance
7
8      def deposit(self, amount):
9          # Add money to the account if the deposit amount is positive
10         if amount > 0:
11             self.balance += amount
12             print(f"Deposited ${amount}")
13         else:
14             # Print a message if the deposit amount is not positive
15             print("Deposit amount must be positive.")
16
17     def withdraw(self, amount):
18         # Withdraw money from the account if there are sufficient funds
19         if amount <= self.balance:
20             self.balance -= amount
21             print(f"Withdrew ${amount}")
22         else:
23             # Print a message if there are not enough funds
24             print("Insufficient funds.")
25
26     def get_balance(self):
27         # Return the current balance of the account
28         return self.balance
29
30 account = BankAccount("John Doe", 1000)
31 account.deposit(500)
32 account.withdraw(200)
33 print(account.get_balance())
```

Output:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Namitha\OneDrive\Desktop\ai\lab 6> & C:/Users/Namitha/AppData/Local/Micros
ha/OneDrive/Desktop/ai/lab 6/task5.py"
Deposited $500
Withdrew $200
1300
PS C:\Users\Namitha\OneDrive\Desktop\ai\lab 6>
```